Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a **hiperparâmetros de arquitetura** e quais correspondem a **hiperparâmetros de treinamento**. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

R: Hiperparâmetro de arquitetura: número de camadas ocultas, número de neurônios e função de ativação.

Hiperparâmetro de treinamento: batch size, epochs, learning rate, otimizador.

Pesos e biases não pertencem a nenhuma dessas duas categorias porque são aprendidos durante o treinamento e representam o que o modelo aprende a partir dos dados.

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

R: Não existe uma regra única para todos os problemas. O resultado vai depender da complexidade do problema e tipo de dado. Mais camadas e neurônios aumentam a capacidade da rede, mas também podem causar overfitting, além de aumentarem o número de parâmetros e o custo computacional. Já uma rede pequena demais pode não conseguir aprender os padrões e gerar underfitting. Por convenção, é comum utilizar potências de 2: 32, 64, 128, 256... neurônios por camada e aumentar ou reduzir a quantidade de neurônios, formando formatos de “V” ou “W”. As funções de ativação trazem a não linearidade e, sem elas, o modelo se comportaria como um modelo linear.

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento,

incluindo quantidade de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

R: Um batch menor, que olha menos amostras por vez, gera mais atualizações por época, mas deixa o gradiente mais ruidoso e o treinamento menos estável. Um batch maior olha para mais amostras por vez, gera menos atualizações por época, porém, um gradiente mais estável.

Um número maior de épocas significa mais passagens completas de todo o conjunto de treinamento pela rede e também mais atualizações de pesos no total. Poucas épocas podem gerar underfitting, já que a rede não aprende os padrões dos dados, e muitas épocas podem gerar overfitting, uma vez que a rede pode se ajustar excessivamente aos dados de treinamento.

O learning rate define o tamanho do passo, ou seja, a magnitude das alterações realizadas nos pesos e biases a cada atualização durante o treinamento. Se for alto, o aprendizado é rápido, mas instável. Se for baixo, o treino é estável, mas lento e com poucas épocas pode causar underfitting.

4) Explique, de forma integrada, o papel da **função de perda, do gradiente, do learning rate e do otimizador** durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

R: A função de perda mede o quanto o modelo está distante do objetivo definido para o treinamento, ou seja, o quanto as previsões da rede estão erradas. O gradiente indica a direção e a intensidade com que cada peso deve mudar para reduzir essa perda. O learning rate define o tamanho do passo dado nessa direção, e o otimizador é o algoritmo responsável por determinar a estratégia de atualização dos parâmetros.

O gradiente não é um hiperparâmetro porque não o escolhemos, ele é calculado automaticamente a cada iteração, a partir dos dados.

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

R: Em uma imagem RGB, cada filtro é formado por uma matriz para cada canal (R, G, e B), com tamanho 3x3x3.

O filtro “desliza” pela matriz da imagem e, em cada posição, multiplica os pesos pelos valores dos pixels da região correspondente em cada canal de entrada. Os resultados são somados, juntamente com o bias, e produzem uma única matriz Z. Após a função de ativação, obtemos o mapa de características. Nesse caso, teremos 16 mapas, que correspondem aos 16 filtros.

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

R: (3x3x3+1) x 16 = 448 parâmetros treináveis.

A quantidade não muda com o tamanho da imagem porque os mesmos filtros são reutilizados em todas as posições. Em uma imagem maior, o filtro só vai deslizar em mais posições.

7) Explique como **stride e padding** influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

R: O stride é o passo do filtro. Com o stride 1, ele anda 1 pixel por vez. Um stride maior pula pixels e gera mapas menores. O padding é a adição de bordas ao redor da imagem.

Com same, o padding é adicionado e com stride 1 a saída tem a mesma altura e largura da entrada. No caso, 32x32 continua 32x32. Com valid, não tem o padding e o filtro só passa onde cabe inteiro e a saída diminui, com filtro de 3x3 e stride 1, o 32x32 viraria 30x30.

8) A primeira camada produz 16 mapas de características, que são recebidos pela segunda camada convolucional. Explique qual será a **profundidade de cada filtro da segunda camada**, quantos mapas de características serão produzidos pelos seus 32 filtros e qual é a função do **Flatten** antes das camadas densas.

R: Como a entrada da segunda camada tem 16 mapas, cada filtro dela terá uma profundidade de 16, ficando com dimensão 3x3x16. Os 32 filtros produzem 32 mapas de características. O Flatten transforma esses mapas em um único vetor, porque as camadas densas só recebem entradas nesse formato.